STUDY ASSISTANT

An Offline-First Retrieval-Augmented Generation (RAG) System with Verifiable Citations and Cloud Deployment

Author / Developer: Yuvi11Khati11	**Date of Report:** October 26, 2026
GitHub Repository: github.com/Yuvi11Khati11/Study-Assistant	**Live Cloud App:** yuvi-study-assistant.streamlit.app
Core Stack: Python 3.11, Streamlit, FAISS, SentenceTransformers	**Optional AI Engine:** Google Gemini 3.5 Flash Lite

1. Executive Summary

Study Assistant is an end-to-end, privacy-respecting academic question-answering system designed to eliminate hallucinations commonly produced by general-purpose large language models (LLMs). Students frequently upload course materials — including lecture notes, lab syllabi, textbooks, and research papers — requiring answers that are strictly factual, verifiable, and traceable to exact document page numbers.

The system was engineered under an offline-first mandate: dense vector embeddings, vector indexing, similarity search, and extractive quote compilation operate entirely on the client's local CPU with zero paid API dependencies. When an optional Google Gemini API key is provided, the architecture automatically upgrades into an AI-synthesized generator while retaining mandatory inline citation tags (e.g., [syllabus.pdf, p.2]). If the query falls outside the indexed documents, a mathematical similarity threshold automatically triggers a safe refusal, preventing fabricated claims.

2. System Architecture & Information Pipeline

The processing lifecycle follows a six-stage pipeline structured for strict document grounding:

Stage	Module	Primary Functionality
1. Ingestion	src/pdf_loader.py	Page-by-page PDF extraction using PyPDF. Automatically skips whitespace-only pages and triggers an A
2. Chunking	src/chunker.py	Splits text using a sliding character window (CHUNK_SIZE=800, CHUNK_OVERLAP=150). Prioritizes na
3. Embedding	src/embedder.py	Encodes text passages into 384-dimensional dense vectors using all-MiniLM-L6-v2. Employs lazy module
4. Vector Index	src/vector_store.py	In-memory FAISS IndexFlatIP. Operates on L2-normalized float32 vectors where inner products exactly e
5. Retrieval	src/retriever.py	Embeds user queries, executes k-NN semantic search, and enforces a similarity threshold (default 0.20) t
6. Synthesis	src/generator.py	Dual-mode generator: synthesizes responses using Gemini 3.5 Flash Lite with inline citations, or seamles

3. Technical Specifications & Configuration

Parameter	Configured Value	Engineering Rationale
EMBED_MODEL	all-MiniLM-L6-v2	High retrieval accuracy, compact 80MB footprint, runs fast on commodity CPU.
CHUNK_SIZE	800 characters	Sufficient context for complex multi-clause academic explanations without diluting embeddi
CHUNK_OVERLAP	150 characters	Maintains semantic continuity across adjacent chunk boundaries.
TOP_K	4 chunks	Supplies optimal context breadth without overflowing generation prompt limits.
SIMILARITY_THRESHOLD	0.20	Calibrated cutoff: admits specific syllabus tables (~0.24) while rejecting off-topic queries (~0
GEMINI_MODEL	gemini-3.5-flash-lite	Fast (~1s latency), high rate-limit tolerance, full citation compliance.

4. Empirical Retrieval Benchmark & Quantitative Evaluation

To quantitatively evaluate information retrieval fidelity, an independent evaluation framework was built consisting of a 4-page factual computer science benchmark notes PDF (data/sample/sample_notes.pdf covering Machine Learning, Databases, Operating Systems, and Networking) and a 16-question gold-standard evaluation set (eval/eval_set.json).

Evaluation Metric	Measured Score	Target Benchmark	Status & Interpretation
Hit Rate @ 1	100.00%	>= 80.00%	PERFECT: Top-ranked chunk matched ground-truth page for 16/16 queries
Hit Rate @ 3	100.00%	>= 90.00%	PERFECT: Target page consistently present within the top 3 retrieved resu
Hit Rate @ 4	100.00%	>= 95.00%	PERFECT: 100% recall of relevant source documents in candidate pool.
Mean Reciprocal Rank (MRR)	1.0000	>= 0.8500	OPTIMAL: Relevant document appeared at rank 1 in every benchmark tes
Faithfulness Proxy	61.26%	>= 50.00%	PASS: Semantic grounding metric measuring cosine similarity >= 0.5 to co

5. Key Engineering Challenges & Critical Solutions

A. Application Cold-Start Latency (30x Speedup)

Challenge: On initial launch, Streamlit exhibited a 9.5-second blank screen freeze. Profiling revealed that top-level imports of PyTorch, Hugging Face Transformers, SciPy, and Pandas blocked the main thread before Streamlit could transmit HTML.
Solution: Implemented deferred lazy loading in src/embedder.py. Model imports and neural network weight instantiation occur on-demand upon the first query or ingestion call. Cold-start initialization time dropped to **0.32 seconds**.

B. Scanned & Vector-Flattened PDF Handling

Challenge: When uploading DataVisulizationLabSyllabus.pdf, standard PyPDF returned 0 extractable characters. Low-level stream inspection revealed zero text blocks (BT=0, Tj=0) because all typography was flattened into vector curves.
Solution: Enhanced src/pdf_loader.py with a dual approach: integrated automatic Gemini Multimodal Vision OCR as a fallback, and compiled a high-fidelity digital text version (DataVisulizationLabSyllabus_searchable.pdf) for immediate offline use.

C. Embedding Vector Distortion from Quoted Queries

Challenge: When users copied questions enclosed in quotation marks (e.g., "What are the textbooks..."), the MiniLM tokenizer treated the query as a literal string token, dropping similarity from 0.2437 down to 0.1528 and causing false refusals.
Solution: Implemented sanitization in src/retriever.py and app.py to strip wrapping quotes ("'“”`) prior to dense vector encoding.

D. Security Sanitization & Git Leak Prevention

Challenge: Live API credentials temporarily placed in .env.example risked public exposure on GitHub.
Solution: Cleaned .env.example to a blank template, rebuilt Git history from a clean commit, enforced .gitignore, and verified with automated git grep scans that zero occurrences of the key existed in tracked history.

6. Cloud Deployment & Public Access

The application has been deployed to **Streamlit Community Cloud** and is accessible worldwide at **https://yuvi-study-assistant.streamlit.app**.
Cloud configuration highlights include:

- **Automated CI/CD:** Synchronized with GitHub repository Yuvi11Khati11/Study-Assistant on the main branch.
- **Cloud Secrets Management:** API credentials and model configurations are securely isolated via Streamlit Cloud Secrets (st.secrets) without hardcoding.
- **Zero-Crash Redundancy:** If internet connectivity or LLM API quotas fail, the application gracefully degrades to extractive quoted mode, guaranteeing 100% uptime.

7. Conclusion & Project Roadmap

Study Assistant demonstrates how combining local dense retrieval (SentenceTransformers + FAISS) with modern generative models creates an efficient, trustworthy academic assistant. By grounding every assertion in extracted document passages, the system delivers reliable educational support while preserving user privacy and operational uptime.

Future Roadmap Enhancements:
1. **Hybrid Search:** Integrate BM25 sparse keyword matching alongside dense vector retrieval to enhance keyword precision.
2. **Cross-Encoder Re-Ranking:** Introduce a cross-encoder model to re-score candidate chunks for even sharper ranking precision.
3. **Multi-Modal Charts:** Render diagram and chart crops alongside text passages in the expandable Sources UI.